



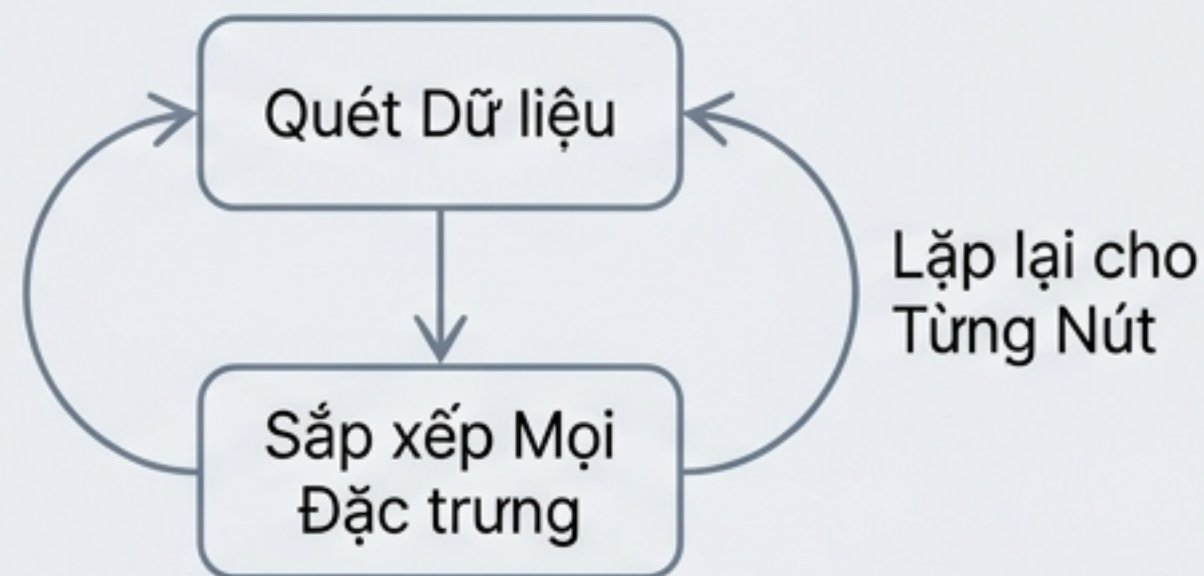
Tối ưu Gradient Boosting trên Dữ liệu lớn (HGB)

Giới thiệu thuật toán Histogram-Based Gradient Boosting trong Scikit-Learn.

BÀI GIẢNG ĐẠI HỌC | MACHINE LEARNING



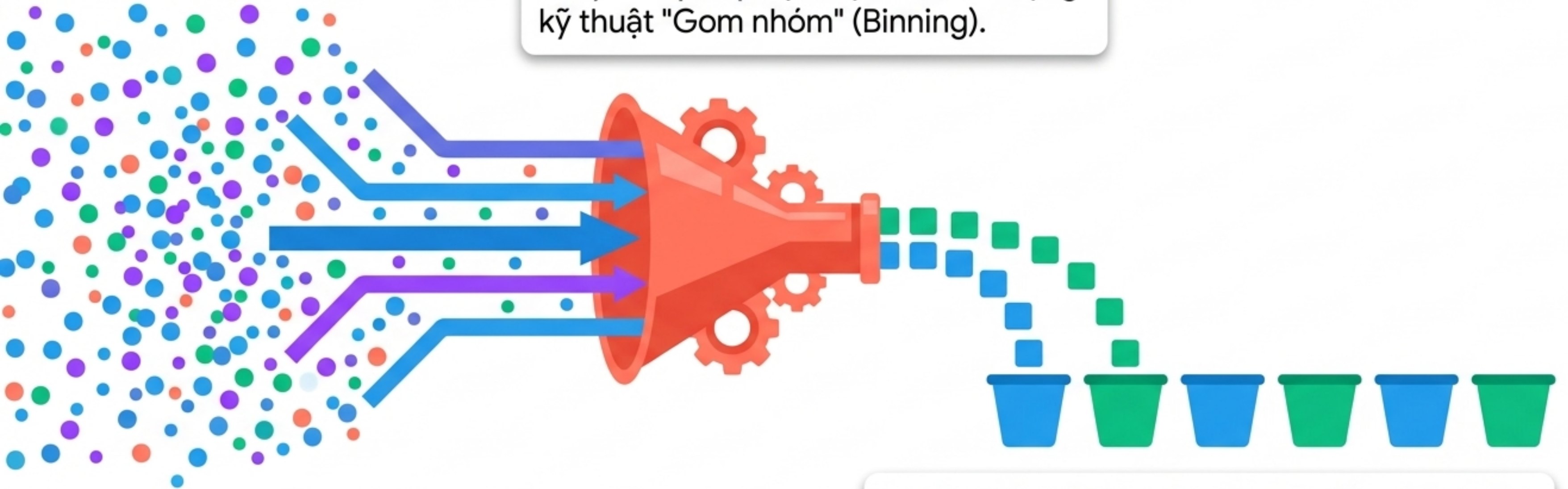
Tại sao GBRT truyền thống lại chậm?



- **Bắt buộc sắp xếp (Sorting):** Để tìm ngưỡng phân chia, GBRT phải liên tục sắp xếp lại toàn bộ giá trị của từng đặc trưng.
- **Chi phí Big-O:** Với độ phức tạp $O(n * m \log(m))$, khi số lượng mẫu (m) lên tới hàng triệu, quá trình sắp xếp ngốn cạn bộ nhớ RAM và thời gian tính toán.

Giải pháp Đột phá: Rời rạc hóa (Binning)

Thay vì sắp xếp cận kề, HGB sử dụng kỹ thuật "Gom nhóm" (Binning).



Chia dải giá trị liên tục đầu vào thành một số lượng **“thùng”** (bins) số nguyên cố định, loại bỏ hoàn toàn bước sắp xếp $m \log(m)$.

1 đến 255

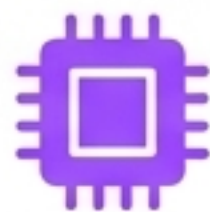
1 đến 255

Bin 256: NaN
(Giá trị thiếu)



1. Phân chia Liên tục

Cắt dải giá trị thực thành các nhóm.



2. Giới hạn 255 Bins

Số nhóm tối đa mặc định là 255, được mã hóa dưới dạng số nguyên 8-bit (uint8).



3. Bin 256 Đặc biệt

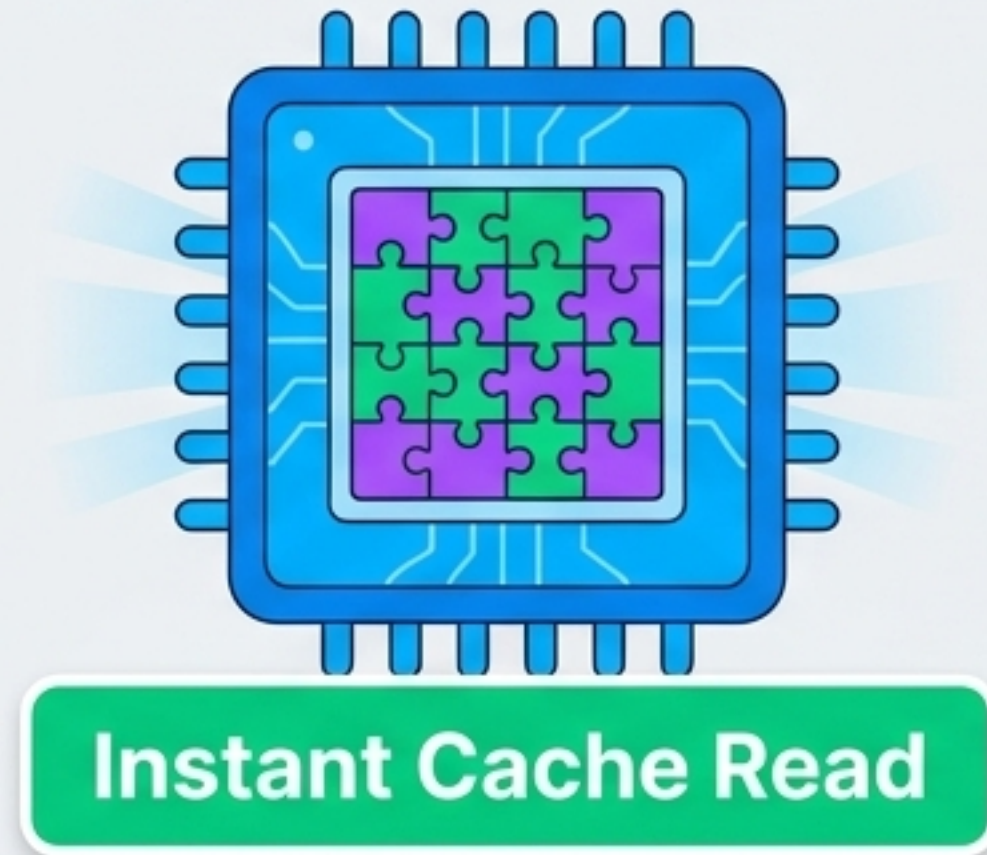
Một bin độc lập dành riêng để chứa toàn bộ các giá trị bị thiếu (Missing values), tránh lỗi mô hình.

Cách cũ: Quét m dòng



Phải duyệt qua hàng triệu điểm dữ liệu thực tế tại mỗi nút. Dữ liệu float64 càng kèngh, gây tràn RAM.

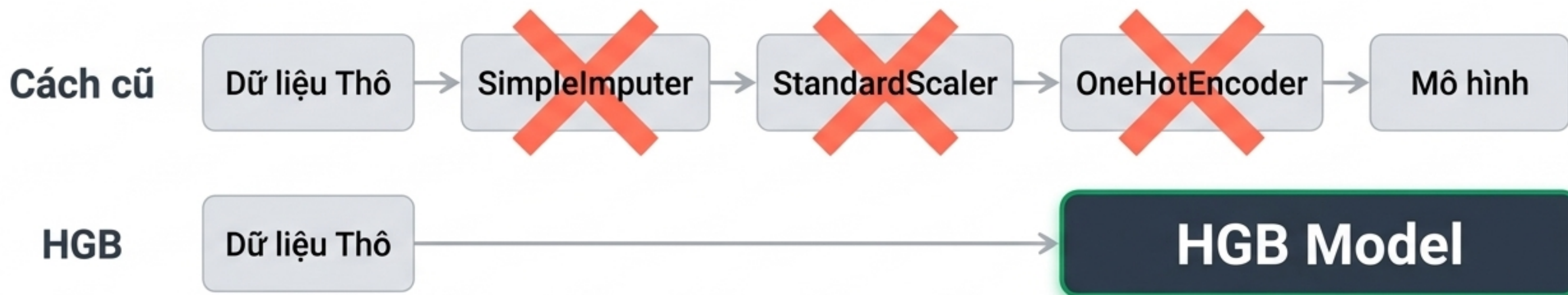
HGB: Quét tối đa 255 Bins



Chỉ duyệt qua biểu đồ tần suất (histogram) tính sẵn. Lưu trữ dạng uint8 thu nhỏ dung lượng RAM đột phá và tối ưu hóa việc đọc bộ nhớ đệm CPU (CPU cache).

	GBRT Truyền thống	HGB
 Độ phức tạp	$O(n * m \log(m))$	$O(b * m)$ (với $b \leq 255$ bins)
 Tốc độ (khi $m > 100.000$)	Rất chậm	Nhanh hơn hàng trăm lần
 Tác động & Độ chính xác	Giữ độ chính xác gốc	Giảm nhẹ (hoạt động như bộ chính quy hóa giúp giảm Overfitting)

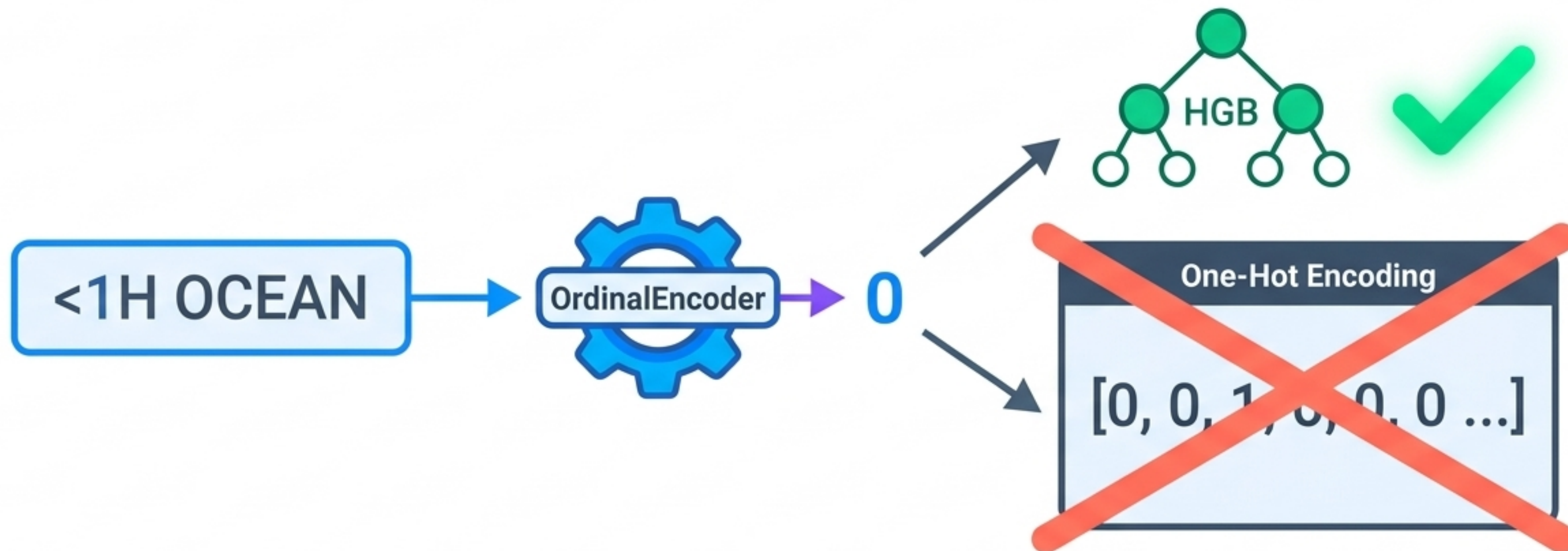
Tối ưu hóa Quy trình Tự động của HGB



Tự xử lý Missing Values: Mô hình tự gán NaN vào Bin 256 và học cách phân nhánh mà không cần SimpleImputer.

Tự động Dừng sớm (Early Stopping): Kích hoạt ngay lập tức nếu tập dữ liệu có $m > 10.000$ mẫu, ngăn ngừa lãng phí tài nguyên.

Xử lý Trực tiếp Dữ liệu Phân loại



Thông qua tham số `categorical_features`, HGB tự động nhận diện các chỉ số cột phân loại.

Thuật toán áp dụng cơ sở phân nhánh dành riêng cho dữ liệu rời rạc, không cần chuyển đổi thành One-Hot Encoding tốn bộ nhớ.

Xây dựng Pipeline Tối ưu cho HGB



Lưu ý: Bỏ qua hoàn toàn bước chuẩn hóa số liên tục (Scaler) và điền khuyết (Imputer).

Cấu hình make_column_transformer cho HGB

```
preprocessing = make_column_transformer(  
    (OrdinalEncoder(), ["ocean_proximity"]),  
    remainder="passthrough"  
)
```

Chuyển đổi chuỗi phân loại (ví dụ: INLAND) thành số nguyên (0, 1, 2...).

Chỉ định rõ tên cột cần xử lý phân loại.

Rất quan trọng: Giữ nguyên trạng thái của tất cả các cột số liên tục khác (không cần Scaler).

Thuyền chỉ toán học hành biết - 16:9

```
hgb_reg = make_pipeline(  
    preprocessing,  
    HistGradientBoostingRegressor(  
        categorical_features=[0],  
        max_bins=255,  
        random_state=42  
    )  
)  
hgb_reg.fit(housing, housing_labels)
```

Báo cho thuật toán biết cột chỉ số 0 là biến phân loại để phân nhánh theo dạng rời rạc.

Giới hạn thùng rác ở mức 255 (Mặc định, lưu dạng 8-bit).

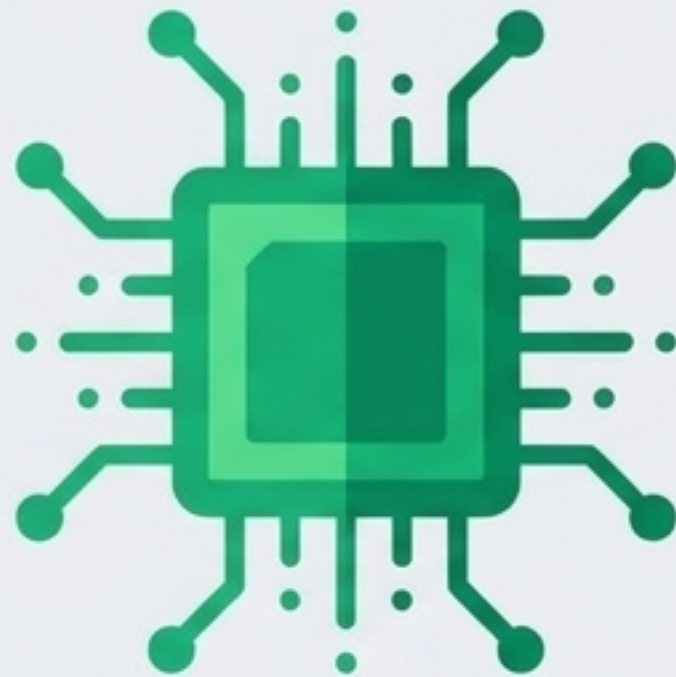
Dữ liệu chui thẳng vào mô hình, tự động xử lý NaN.

Tham số `n_estimators` cũ nay được đổi tên thành `max_iter` trong HGB.



Tốc độ Vượt trội

Loại bỏ hoàn toàn nút thắt sắp xếp dữ liệu, duyệt qua biểu đồ Histogram giúp mô hình nhanh hơn hàng trăm lần trên Big Data.



Siêu Tiết kiệm RAM

Gom nhóm dữ liệu về định dạng 8-bit (uint8) tối ưu tuyệt đối bộ nhớ và tốc độ đọc CPU Cache.



Tối giản Pipeline

Kiến trúc thông minh tự động xử lý giá trị khuyết (NaN), Dừng sớm (Early Stopping) và dữ liệu Phân loại.



Bước tiếp theo: Vượt qua Giới hạn

Bạn đã làm chủ tốc độ của HGB. Vậy làm sao để tối ưu hóa độ chính xác lên mức tối đa?

Khám phá kỹ thuật **Phân lớp xếp chồng (StackingClassifier)** để kết hợp sức mạnh của HGB với các mô hình Machine Learning khác trong bài giảng tới.